

Case Study: Intermediate Code Generation for Arithmetic Expressions

Student Name: AFEEF AHMED SHAIK

Reg. No. RA2311026050166

Course: Compiler Design

Subject Code: 21CSC304J

1. Title:

Arithmetic Expression Processing using Intermediate Representation (Prefix, Postfix and Syntax Tree Construction).

2. Aim

The aim of this project is to design and implement a system that converts arithmetic expressions from infix notation into postfix and prefix formats and constructs a syntax tree to represent the expression in a structured form.

3. Objectives

- To accept and process arithmetic expressions provided in infix form
- To apply operator precedence rules correctly during conversion
- To display the structure of the expression using a tree representation
- To convert infix expressions into postfix notation using stack operations
- To generate prefix notation from the given expression
- To build a syntax tree representing the expression hierarchy.

4. Background and Problem Description

In compiler design, source code written by programmers must be transformed into a form that can be easily processed by machines. Arithmetic expressions are typically written in infix notation, which is convenient for humans but not suitable for direct evaluation by compilers.

To simplify processing, compilers convert expressions into alternative formats such as postfix and prefix notations. These formats eliminate the need for parentheses and clearly define the order of operations.

However, these linear representations do not fully capture the structural relationships within an expression. To overcome this limitation, a syntax tree is used. The syntax tree provides a hierarchical representation where operators and operands are organized based on precedence and associativity.

This case study focuses on implementing these transformations and constructing a syntax tree to better understand intermediate code generation.

5. Key Concepts

A. Expression Notations

- **Infix Notation:** Operators are placed between operands (e.g., $a + b$).
- **Prefix Notation:** Operators precede operands (e.g., $+ab$).
- **Postfix Notation:** Operators follow operands (e.g., $ab+$).

These forms help eliminate ambiguity and are widely used in compilers.

B. Syntax Tree

A syntax tree is a binary tree structure used to represent expressions.

- Internal nodes represent operators
- Leaf nodes represent operands

This structure ensures that the order of operations is preserved and allows easier evaluation and analysis.

C. Operator Precedence

Operators are evaluated based on priority:

- Highest $\rightarrow$ $*$, $/$
- Medium $\rightarrow$ $+$, $-$
- Lowest $\rightarrow$ parentheses

This priority is handled during conversion using a stack.

6. Numerical Worked Example

Given Expression

$(a+b)*(c-d)$

Step 1: Postfix Conversion

$ab+cd-*$

Explanation:

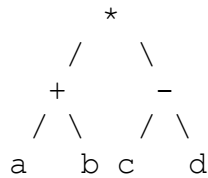
The operators are placed after operands, removing the need for parentheses.

Step 2: Prefix Conversion

$ab+cd-*$

The operators are placed after operands, removing the need for parentheses

Step 3: Syntax Tree Representation



Explanation:

This tree shows:

- * as the root
- + and - as intermediate operators

7. Algorithm

1. Read the input expression in infix form
2. Scan each character of the expression
3. If the character is an operand, add it to output
4. If the character is an operator:
 - Compare precedence with stack top
 - Push or pop accordingly
5. Handle parentheses appropriately
6. Generate postfix expression
7. Reverse the infix expression and apply similar logic to obtain prefix
8. Construct syntax tree using postfix expression:
9. Push operands as nodes
10. For operators, create a node and attach two children
11. Display all results

8. Implementation Details

- The program is implemented in **C language**
- A stack is used for handling operators during conversion
- Arrays are used for storing expressions
- A binary tree structure is used for syntax tree creation
- Dynamic memory allocation is used for node creation
- The syntax tree is displayed using a structured console format the original infix expression with parentheses

9. Results and Discussion

The program was tested with various arithmetic expressions. For the input $(a+b)*(c-d)$, the System correctly generated:

- Postfix form: $ab+cd-*$
- Prefix form: $*+ab-cd$

The syntax tree produced correctly represents the structure of the expression, maintaining proper operator precedence and associativity.

The results confirm that the stack-based conversion and tree construction methods are effective and reliable for intermediate representation.

10. Conclusion

This case study successfully demonstrates the process of transforming arithmetic expressions into different intermediate representations, namely postfix and prefix notations, along with the construction of a syntax tree. These transformations play a crucial role in compiler design, as they simplify the evaluation process and eliminate ambiguity present in infix expressions. The implementation using stack-based algorithms ensures that operator precedence and associativity are handled correctly during conversion. The generated postfix and prefix expressions clearly reflect the intended order of operations, making them suitable for further stages of compilation such as code generation and optimization.

The syntax tree constructed from the postfix expression provides a hierarchical representation of the expression. It clearly illustrates the relationship between operators and operands, where each internal node represents an operator and each leaf node represents an operand. This structured representation is essential for understanding how compilers internally process expressions. Additionally, the project highlights the practical application of fundamental data structures such as stacks and trees in solving real-world problems in compiler design. The implementation is efficient, easy to understand, and demonstrates how theoretical concepts are applied in practice.

Overall, this case study provides a clear insight into intermediate code generation and emphasizes the importance of structured representations in simplifying complex computations. It serves as a foundational step toward understanding more advanced topics in compiler construction, such as parsing, optimization, and code generation.

